

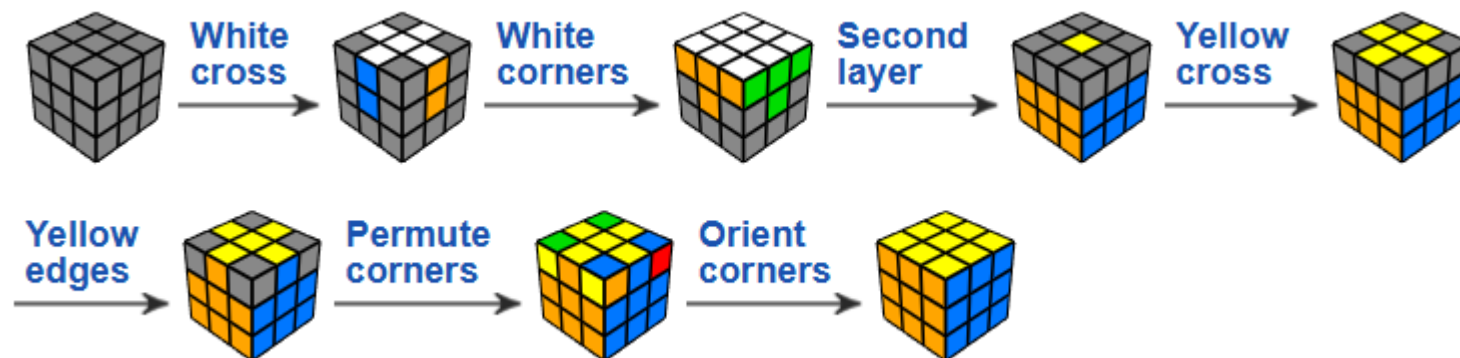


DSA103 Advanced Chemical Data Science

Lecture 2: Computational Thinking
(with Python)

Prof. Meredith C. Schuman

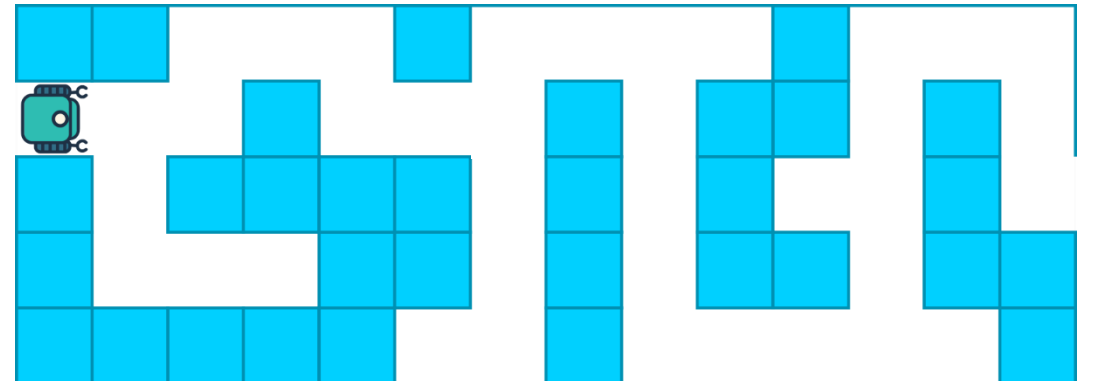
meredithchristine.schuman@uzh.ch



Discussion of homework

Swap your pseudocode with your neighbor and be each other's “robots”!
Afterwards, we will discuss together.

As the robot, does the pseudocode make you smart enough to solve the maze?
If so, how?
If not, why not?



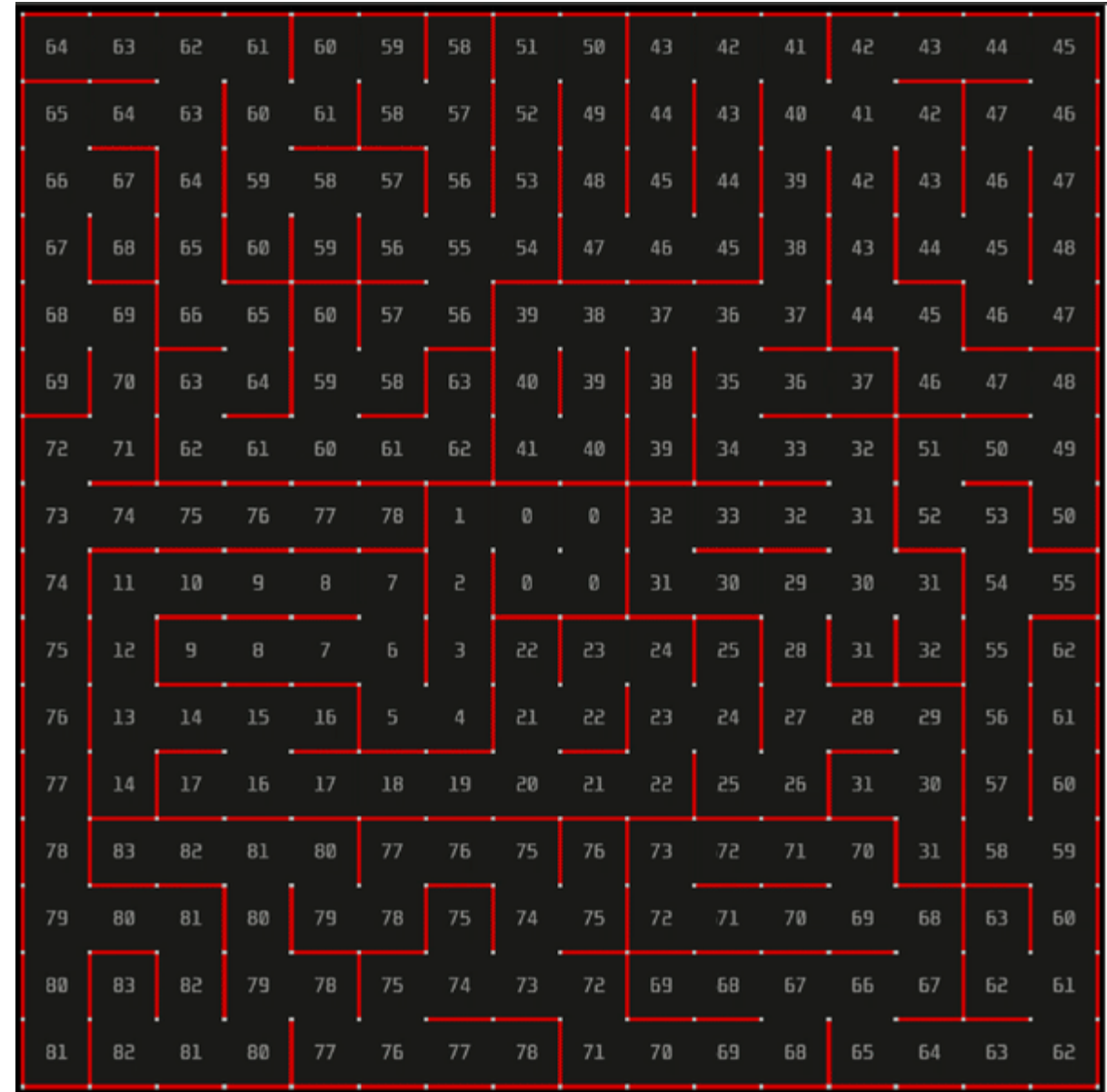
A solution

The wall follower...

- might not find the exit under all conditions, mazes
- definitely won't find the optimal solution

Finding the optimal solution is a complete branch of computer science.

See e.g. Generating and Solving a Perfect Maze (With Algorithms)



<https://github.com/darshit-desai/Maze-Solver-simulation-using-Wall-Following-Algorithm-OOP>

From pseudocode to real code to Python

To grasp a high-level problem, we used simplified language and pseudocode:

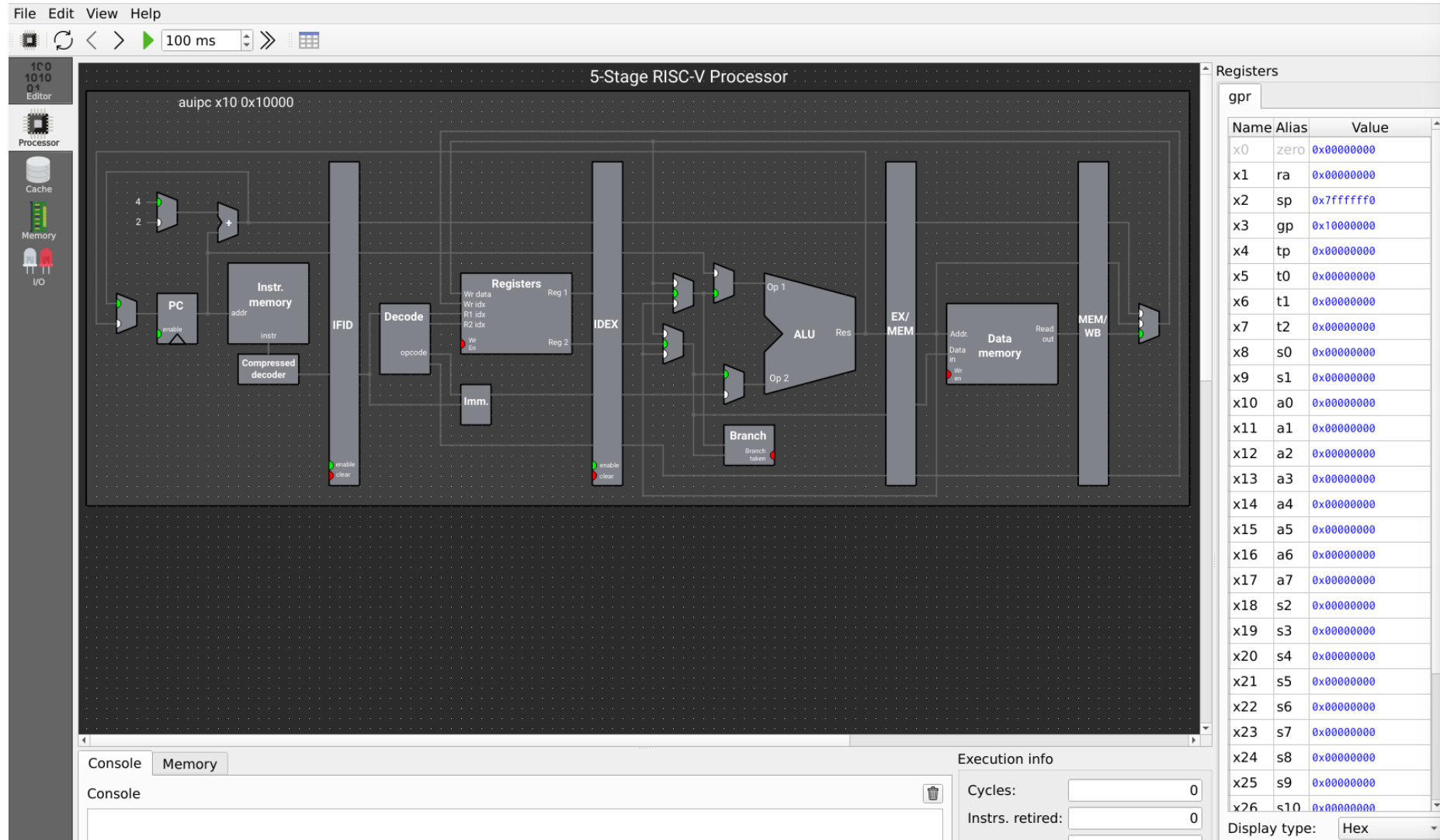
```
function rotate_left
```

```
    rotate_clockwise; rotate_clockwise; rotate_clockwise
```



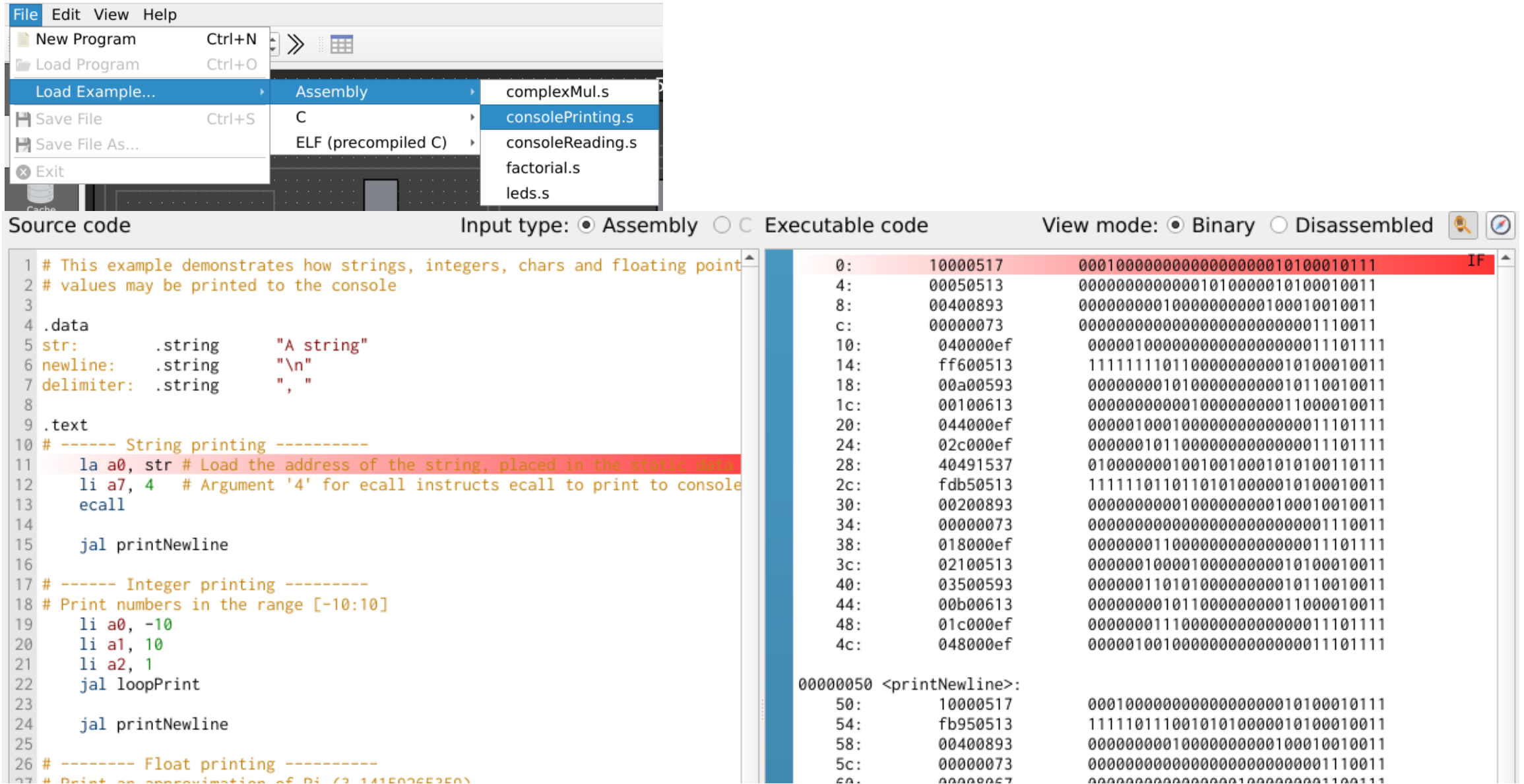
Internally, the machine executing our instructions operates on a much lower level.

From pseudocode to real code to Python



From ripes.me in 2025 (site no longer up), running <https://github.com/Varunkumar0610/RISC-V-32I-5-stage-Pipeline-Core>

From pseudocode to real code to Python



From pseudocode to real code to Python

Source code

Input type: ☒ Assembly ☐ Executable code

View mode: ☒ Binary ☐ Disassembled

```
1 # This example demonstrates how strings, integers, chars and floating point
2 # values may be printed to the console
3
4 .data
5 str:      .string      "A string"
6 newline:  .string      "\n"
7 delimiter: .string      ", "
8
9 .text
10 # ----- String printing -----
11     la a0, str # Load the address of the string, placed in the static data
12     li a7, 4   # Argument '4' for ecall instructs ecall to print to console
13     ecall
14
15     jal printNewline
16
17 # ----- Integer printing -----
18 # Print numbers in the range [-10:10]
19     li a0, -10
20     li a1, 10
21     li a2, 1
22     jal loopPrint
23
24     jal printNewline
25
26 # ----- Float printing -----
27 # Print an approximation of Pi (3.14159265359)
28     li a0, 0x40490FDB
29     li a7, 2
30     ecall
31
32     jal printNewline
33
34 # ----- ASCII character printing -----
35 # Print ASCII characters in the range [33:53]
36     li a0, 33
```

0: 10000517 00010000000000000000000010100010111

4: 00050513 00000000000000001010000010100010011

8: 00400893 00000000010000000000100010010011

c: 00000073 0000000000000000000000000110011

10: 040000ef 000001000000000000000000011101111

14: ff600513 1111111011000000000010100010011

18: 00a00593 000000001010000000000010110010011

1c: 00100613 00000000000100000000011000010011

20: 044000ef 000001000100000000000000011101111

24: 02c000ef 00000010110000000000000001101111

28: 40491537 01000000010010010001010100110111

2c: fdb50513 11111101101101010000010100010011

30: 00200893 000000000100000000000000010010011

34: 00000073 0000000000000000000000000110011

38: 018000ef 000000011000000000000000011101111

3c: 02100513 0000010000100000000010100010011

40: 03500593 00000011010100000000010110010011

44: 00b00613 00000000101100000000011000010011

48: 01c000ef 000000011100000000000000011101111

4c: 048000ef 000001001000000000000000011101111

00000050 <printNewline>:

50: 10000517 00010000000000000000000010100010111

54: fb950513 11111011100101010000010100010011

58: 00400893 00000000010000000000100010010011

5c: 00000073 0000000000000000000000000110011

60: 00008067 000000000000000001000000001100111

00000064 <loopPrint>:

64: 00050293 000000000000001010000001010010011

68: 00058313 000000000000001011000001100010011

0000006c <loop>:

6c: 00028513 00000000000000101000010100010011

70: 00060893 000000000000001100000100010010011

74: 00000073 0000000000000000000000000110011

gpr

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x00000000
x6	t1	0x00000000
x7	t2	0x00000000
x8	s0	0x00000000
x9	s1	0x00000000
x10	a0	0x10000000
x11	a1	0x00000000
x12	a2	0x00000000
x13	a3	0x00000000
x14	a4	0x00000000
x15	a5	0x00000000
x16	a6	0x00000000
x17	a7	0x00000004

Console

Memory

Console

A string (After 7 steps)

Printing "A string" in Python

print("A string")

Every higher-level language is executing machine instructions in the end (including Python).

From ripes.me in 2025 (site no longer up), running <https://github.com/Varunkumar0610/RISC-V-32I-5-stage-Pipeline-Core>

From pseudocode to real code to Python

Syntax, Semantics, Types

We define a certain structure so that it is easier to read by humans and uniquely defines what the computer will execute and calculate.

Syntax

Semantics

Types

Often, but not always, a mix of English and mathematics:

```
a = 2
b = 3
f = a + b
f is 5
```

```
if (a > b)
```

```
while (condition)
```

Exercises on GitHub

Please fork and clone the repository by Friday. Come with any questions!

(See README; you'll need a free GitHub account.)

[Link to helpful GitHub tutorials](https://github.com/mcschuman/ChemicalDataScience103)

<https://github.com/mcschuman/ChemicalDataScience103>

The screenshot shows the GitHub repository page for 'ChemicalDataScience103'. At the top, the repository name is displayed with a 'Public' badge. Below this, there are buttons for 'main' (selected), '1 Branch', and '0 Tags'. A search bar labeled 'Go to file' is present, along with 'Add file' and 'Code' buttons. The commit history shows a single commit by 'mcschuman' titled 'chore: initial commit' with hash '624ecf0' and timestamp '14 minutes ago'. Below the commit, a list of files is shown: 'assignments', 'lectures', '.gitignore', 'README.md', 'pyproject.toml', and 'uv.lock', each with its commit hash and timestamp. The 'README' file is selected, showing its content. The README title is 'DSA103'. The content describes the repository for the course 'DSA103 Advanced Chemical Data Science', lists the instructors (Prof. Meredith C. Schuman and Dr. Johannes Schörghener) with their email addresses, and mentions the teaching assistant (Dimitrios Xynos) with his email address. The section 'Prerequisites' is also visible at the bottom.

ChemicalDataScience103 Public

main 1 Branch 0 Tags

Go to file T Add file <> Code

mcschuman chore: initial commit 624ecf0 · 14 minutes ago 2 Commits

assignments	chore: initial commit	14 minutes ago
lectures	chore: initial commit	14 minutes ago
.gitignore	started	32 minutes ago
README.md	chore: initial commit	14 minutes ago
pyproject.toml	chore: initial commit	14 minutes ago
uv.lock	chore: initial commit	14 minutes ago

README

DSA103

Repository for the course DSA103 Advanced Chemical Data Science. All course material will be shared here, whereas OLAT will only be used for all course-related communication.

Instructors and persons responsible for the course: Prof. Meredith C. Schuman (meredithchristine.schuman@uzh.ch), Dr. Johannes Schörghener (johannes.schoergenheimer@chem.uzh.ch)

Teaching assistant: Dimitrios Xynos (dimitrios.xynos@chem.uzh.ch)

Prerequisites

Concepts we will need for the exercises

What is a string?

Data type

Sequence of characters enclosed in quotes

What can you do with a string?

Iterate (loop)

Identify the nature of the characters (e.g. letter, number)

Index

Slice

Alter contents

Convert to a different type

What is a regular expression (RegEx)?

Sequence of characters specifying a matching pattern in strings

Regular expressions are not required for these exercises but may make for easier solutions.

Requires `import re` in Python

Use <https://regex101.com/> to design RegEx

Exercise 1: Molecular Formula Parser

Objective: Write a function `parse_molecular_formula(formula)` that counts atoms in molecular formulas. Start with the simple cases, but design your solution to handle increasingly complex inputs without major rewrites.

Think computationally: How can you break this problem into smaller, reusable pieces that won't need to be completely rewritten when the requirements change?

```
# Simple single-letter elements, single-digit counts
test_cases_level1 = [
    ("H2O", {"H": 2, "O": 1}),
    ("CH4", {"C": 1, "H": 4}),
    ("CO2", {"C": 1, "O": 2}),
    ("NH3", {"N": 1, "H": 3})
]
```

```
# Two-letter elements like Ca, Cl, Na
test_cases_level2 = [
    ("NaCl", {"Na": 1, "Cl": 1}),
    ("CaCl2", {"Ca": 1, "Cl": 2}),
    ("MgO", {"Mg": 1, "O": 1}),
    ("AlCl3", {"Al": 1, "Cl": 3}),
    ("H2SO4", {"H": 2, "S": 1, "O": 4})
]
```

```
# Multi-digit counts
test_cases_level3 = [
    ("C12H22O11", {"C": 12, "H": 22, "O": 11}), # Sucrose
    ("Ca10P6O26", {"Ca": 10, "P": 6, "O": 26}), # Hydroxyapatite
    ("C2H5OH", {"C": 2, "H": 6, "O": 1}),       # Ethanol
    ("Fe2O3", {"Fe": 2, "O": 3})                # Iron oxide
]
```

Exercise 2: Chemical Equation Balancer Checker

Objective: Write a program that checks if a chemical equation is balanced.

Problem: Create a function called `is_balanced()` that takes a chemical equation string and returns True if balanced, False if not.

Your Task:

- Split the equation at the arrow symbol "→" to get reactants and products
- For each side, parse the molecules and their coefficients
- Count total atoms of each element on both sides
- Compare the totals

Example Equation Format: "2H₂ + O₂ → 2H₂O"

Exercise 3: Reaction Pathway Analyzer

Objective: Analyze multi-step reaction sequences to find synthesis pathways.

Problem: Write a program that determines what compounds can be synthesized from starting materials through a series of reactions, and finds a pathway to create a target compound.

Your Task: Create a function called `find_synthesis_path()` that takes:

- A list of starting compounds
- A list of reaction equations
- A target compound to synthesize

The function should return either:

- A list of reaction steps that lead to the target compound, OR
- A message saying the target cannot be synthesized

Let's try!

- Implement UV and start using UV environments
- Your assignment is in session02_assignment. Show your reasoning in comments and/or Markdown fields.

What you don't finish today in class will be **homework**. Ask questions and help each other.

Next session (Wednesday), please raise questions that came up while attempting the homework.

We will discuss solutions in class next Friday. The **homework must be submitted by 7:59 next Friday**.

Submit homework to dimitrios.xynos@chem.uzh.ch, subject "DSA103 Yourname".

